How Hessian and Sharpness Work in the Raman-Suppression Codebase

A code-grounded pedagogical note

Prepared from the repository state on 2026-04-23

April 23, 2026

Abstract

This note explains, in a code-grounded and pedagogical way, what the *Hessian* and *sharpness* mean in this repository, how they are computed, when each one is used, and how they relate to the base Raman suppression optimization pipeline.

The short version is:

1. The code's **Hessian machinery** is mainly a *diagnostic* tool. It is used to probe local curvature of the physical loss near an already-optimized phase, mainly through Hessian-vector products and eigenvalue calculations.
2. The code's **sharpness machinery** is mainly an *optimization regularizer*. It adds a stochastic estimate of local curvature to the objective so that the optimizer prefers flatter, more robust basins rather than merely deeper ones.

These two mechanisms are related, but they are not the same thing. The Hessian path tries to understand the full local geometry. The sharpness path uses a cheaper trace-like summary of that geometry inside the objective.

One caveat is important: the default production optimizer is L-BFGS and does not use Hessian/Newton steps, but the repository also contains a separate research trust-region optimizer that uses finite-difference Hessian-vector products during optimization.

1 Roadmap

The relevant implementation lives in these main files:

1. `scripts/lib/raman_optimization.jl`
   Base physical cost and adjoint gradient.
2. `scripts/research/phases/phase13/hvp.jl`
   Matrix-free finite-difference Hessian-vector products.
3. `scripts/research/phases/phase13/hessian_eigspec.jl`
   Eigenspectrum extraction from those HVPs.
4. `scripts/lib/sharpness_optimization.jl`
   Sharpness-aware objective and optimizer wrapper.
5. `scripts/research/trust_region/trust_region_optimize.jl`
   Separate Newton-style trust-region research optimizer that uses HVPs inside the optimization loop.

2 The Three Places Curvature Appears

The easiest way to avoid confusion is to separate three code paths:

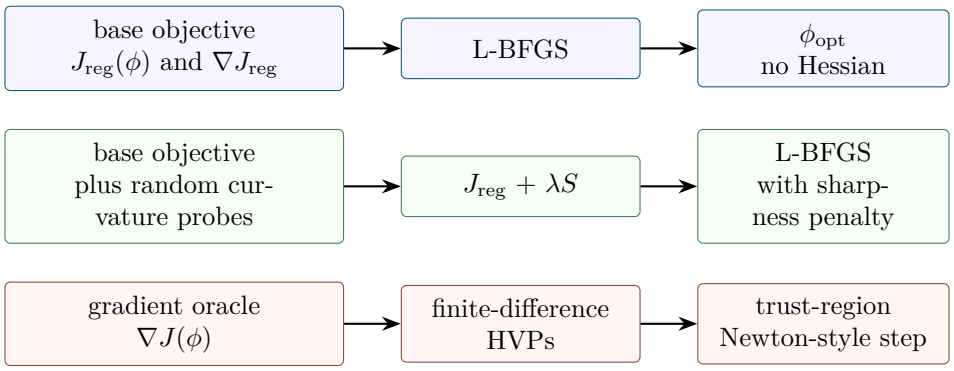


Figure 1: Three separate paths in the repo. The default optimizer is first-order L-BFGS. Sharpness modifies the objective but still usually uses L-BFGS. The trust-region research path uses HVPs as part of a Newton-style optimizer.

Practical Takeaway

When someone says "the Hessian code," ask which path they mean. In this repository, Hessian-vector products are mostly diagnostic, sharpness is a regularizer, and trust-region Newton is a separate research optimizer.

3 The Base Quantity Being Differentiated

Everything starts from the same underlying function:

$$J(\phi),$$

where ϕ is the spectral phase applied to the input pulse.

In code, the base forward-adjoint computation is implemented in `cost_and_gradient(...)` in `scripts/lib/raman_optimization.jl`. The pipeline is:

1. apply the spectral phase to the input field,
2. propagate the field through the fiber,
3. evaluate the Raman-band cost at the output,
4. solve the adjoint problem backward,
5. apply the chain rule to get $\nabla_\phi J$.

Schematically, the code computes

$$\phi \longmapsto u_{\omega,0} e^{i\phi} \longmapsto \text{fiber propagation} \longmapsto J(\phi),$$

and then returns both the scalar objective and its gradient with respect to ϕ.

3.1 Regularized and log-scaled objective

The code can optimize either a linear-scale objective or a dB-scaled one, and it can add regularization terms such as GDD and boundary penalties. So the actual optimized surface is often

$$J_{\text{reg}}(\phi) = J_{\text{physics}}(\phi) + \lambda_{\text{gdd}} R_{\text{gdd}}(\phi) + \lambda_{\text{boundary}} R_{\text{boundary}}(\phi),$$

possibly followed by a log transform.

Practical Takeaway

Whenever this note says "Hessian" or "sharpness," it always means the second-order geometry of *whatever loss function the code is currently using.* In some paths that is the plain linear physical loss. In other paths that is the dB-scaled, regularized loss.

4 What the Hessian Means Here

Mathematically, the Hessian is the matrix of second derivatives:

$$H(\phi) = \nabla_\phi^2 J(\phi).$$

If the gradient tells you the local slope, the Hessian tells you the local curvature:

- positive curvature means "bowl-shaped" in that direction,
- negative curvature means "saddle" or "downhill escape" direction,
- near-zero curvature means "flat" direction.

For a perturbation v, the Hessian-vector product

$$H(\phi)v$$

tells you how the gradient changes if you move infinitesimally in direction v.

4.1 Why the code does not build the full Hessian by default

The phase dimension is large. On production grids the optimization variable can have thousands of components, so explicitly building the dense Hessian matrix would be expensive in both time and memory.

Instead, the repo usually works *matrix-free*: it computes

$$v \mapsto H(\phi)v$$

without ever forming the full matrix.

In plain language, that means the code knows how the Hessian acts on a vector, without ever writing down every entry of the Hessian itself.

5 How the Hessian-Vector Product is Computed

The implementation in `scripts/research/phases/phase13/hvp.jl` uses a central finite difference of the gradient:

$$H(\phi)v \;\approx\; \frac{\nabla J(\phi+\varepsilon\hat{v}) - \nabla J(\phi-\varepsilon\hat{v})}{2\varepsilon}\,\|v\|\,, \qquad \hat{v} = \frac{v}{\|v\|}.$$

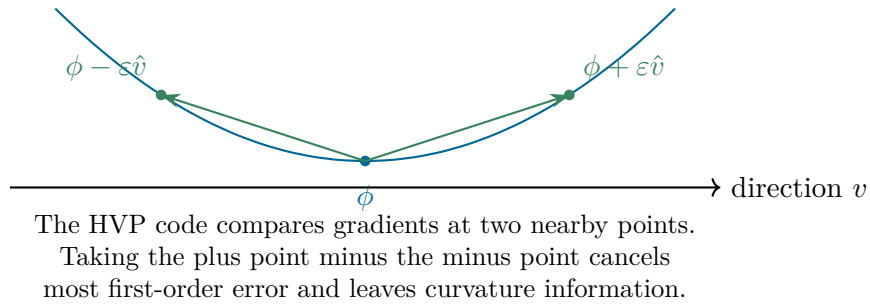


Figure 2: Central finite difference picture for a Hessian-vector product.

5.1 Why the direction is normalized

The code first replaces v by $\hat{v} = v/\|v\|$, perturbs by a fixed small step $\varepsilon\hat{v}$, then rescales by $\|v\|$ at the end.

This is numerically important. Without normalization, a large-norm v would produce an enormous perturbation εv, and a tiny-norm v would produce an almost invisible perturbation. The normalization makes the finite difference step size controlled by ε, not by the arbitrary scale of v.

5.2 Cost of one HVP

One HVP requires two gradient evaluations:

1. $\nabla J(\phi + \varepsilon\hat{v})$,
2. $\nabla J(\phi - \varepsilon\hat{v})$.

Since each gradient evaluation itself requires a forward solve plus an adjoint solve, one HVP is already a nontrivial operation. That is another reason the code prefers matrix-free methods.

6 How the Hessian Eigenspectrum is Computed

Once the code can apply H to arbitrary vectors, it wraps that HVP in a matrix-free linear operator and gives it to `Arpack.eigs`.

6.1 What is extracted

The eigenspectrum script computes:

- the biggest positive-curvature eigenvalues, using `:LR`,
- the most negative or smallest eigenvalues, using `:SR`.

So it is not trying to recover the entire spectrum. It is trying to recover the two most informative wings:

1. the stiff positive-curvature directions,
2. the soft or negative-curvature directions.

6.2 Interpretation

Suppose the code finds

$$\lambda_{\max} > 0 \qquad \text{and} \qquad \lambda_{\min} < 0.$$

Then the point is *indefinite*: in simpler words, it is not a true minimum. It is a saddle, meaning some directions curve up while other directions curve down.

That was the key Phase 13 geometry result: the canonical L-BFGS optima were not true minima. They had negative-curvature directions that first-order L-BFGS was not exploiting.

Key Idea

The Hessian path answers the question: *"What kind of critical point did the optimizer land on?"*
In the Phase 13 code path it is a geometry diagnosis tool, not primarily an optimizer.

6.3 Small-N exact path

For small problems only, the repo also has a dense cross-check path:

$$H = \begin{bmatrix} He_1 & He_2 & \cdots & He_N \end{bmatrix},$$

where each column is built from the HVP routine applied to a basis vector e_k. The code only allows this for small dimension because it scales like $2N$ gradient evaluations.

7 The Separate Trust-Region Newton-Style Optimizer

There is one important exception to the "Hessian is post-hoc" rule. The repo also has a research optimizer in `scripts/research/trust_region/trust_region_optimize.jl`.

That optimizer is Newton-style because it chooses steps using a local quadratic model

$$m(p) = J(\phi) + \nabla J(\phi)^\top p + \frac{1}{2} p^\top H(\phi) p,$$

inside a trust region

$$\|p\| \le \Delta.$$

But it still does *not* build the dense Hessian matrix. Instead, inside each iteration it defines an operator

$$v \mapsto H(\phi)v,$$

and computes that HVP by finite-differencing gradients, just like the Phase 13 diagnostic HVP path. The trust-region version chooses its finite-difference step adaptively instead of using the fixed Phase 13 default.

Practical Takeaway

So the answer is: **the default L-BFGS optimizer does not use Hessian/Newton steps, but the separate trust-region research optimizer does use finite-difference HVPs inside optimization.**

There is also an experimental `strategy = :newton` option in the sharpness wrapper, `scripts/lib/sharpness_optimization.jl`. In the checked code paths, the actual sharpness drivers use the default L-BFGS strategy, so this option should not be confused with the trust-region HVP optimizer.

8 What Sharpness Means Here

The repo's sharpness implementation is in `scripts/lib/sharpness_optimization.jl`. It is *not* a full Hessian eigendecomposition inside the optimizer.

Instead, it builds a stochastic curvature summary:

$$S(\phi).$$

The word *summary* is important. The full Hessian contains a huge amount of information:

- one curvature value for every direction,
- one eigenvalue for every eigenvector,
- detailed information about how uneven the basin is in different directions.

Sharpness throws most of that information away on purpose and keeps only a single scalar that answers:

"On average, how curved is the loss near this point?"

So the stochastic sharpness estimator is a way to compress a large curvature object into one number that is cheap enough to use during optimization.

The implemented estimator is

$$S(\phi) \approx \frac{1}{N_s} \sum_{i=1}^{N_s} \frac{J(\phi + \varepsilon P v_i) + J(\phi - \varepsilon P v_i) - 2J(\phi)}{\varepsilon^2},$$

where:

- v_i are Rademacher vectors with entries ± 1,
- P is a projector that removes the two gauge directions,
- ε is the finite-difference step,
- N_s is the number of stochastic probe vectors.

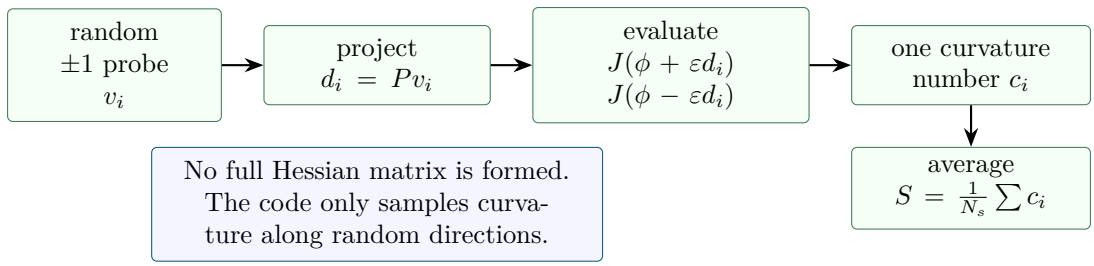


Figure 3: Sharpness is a random average of many one-dimensional curvature measurements.

8.1 Step 1: directional curvature from Taylor expansion

Take any direction d. A second-order Taylor expansion gives

$$J(\phi + \varepsilon d) = J(\phi) + \varepsilon \nabla J(\phi)^\top d + \frac{\varepsilon^2}{2} d^\top H(\phi) d + \mathcal{O}(\varepsilon^3),$$

and similarly

$$J(\phi - \varepsilon d) = J(\phi) - \varepsilon \nabla J(\phi)^\top d + \frac{\varepsilon^2}{2} d^\top H(\phi) d + \mathcal{O}(\varepsilon^3).$$

If we add these two expansions, the linear terms cancel:

$$J(\phi + \varepsilon d) + J(\phi - \varepsilon d) - 2J(\phi) = \varepsilon^2 d^\top H(\phi) d + \mathcal{O}(\varepsilon^4).$$

Dividing by ε^2 gives

$$\frac{J(\phi + \varepsilon d) + J(\phi - \varepsilon d) - 2J(\phi)}{\varepsilon^2} = d^\top H(\phi) d + \mathcal{O}(\varepsilon^2).$$

So the symmetric finite difference in the code is estimating the *directional second derivative* in direction d.

Key Idea

Each probe asks: *"If I move a little in this direction and a little in the opposite direction, how bowl-shaped or saddle-shaped is the loss along that line?"*

8.2 Step 2: from one directional curvature to an average curvature

Now suppose we do not want the curvature in just one carefully chosen direction. We want an *average* over many directions. Then we can sample random probe vectors v and average

$$v^\top H(\phi) v.$$

Using random probes is what makes the construction *stochastic*. Instead of summing over a huge set of directions, which would be expensive, the code uses a small random set and treats the resulting average as a Monte Carlo estimate of a larger average.

8.3 What a Hutchinson-style estimator is, in plain language

Suppose I ask you to find the average height of every student in a huge university. One method is to measure every student. That is exact, but slow. Another method is to sample a few hundred students at random and average those. That is not exact, but it is much cheaper, and if the sample is chosen well it can be very accurate.

A Hutchinson-style estimator is the same kind of idea, but for matrix quantities instead of people.

Here the expensive quantity is the trace of the Hessian, which is the sum of all diagonal entries:

$$\mathrm{tr}(H) = H_{11} + H_{22} + \cdots + H_{NN}.$$

If we had the full Hessian matrix in memory, we could compute that sum directly. But in this project we usually do *not* have the full Hessian. We only know how to ask questions of the form

$$v \mapsto Hv.$$

So the problem is:

> How can we estimate a big quantity like $\mathrm{tr}(H)$ using only cheap-ish random probes, without ever building the full matrix?

Hutchinson's answer is:

1. pick a random vector v,
2. compute the scalar $v^\top Hv$,
3. repeat a few times,
4. average the results.

That average is the estimator.

The word *style* here just means the repo is using the same basic mathematical trick, even though it wraps it in finite differences and gauge projection for this particular physics problem.

Practical Takeaway

A Hutchinson estimator is just a random-sampling trick for estimating a large matrix summary, usually a trace, without constructing the whole matrix.

8.4 Step 3: why this becomes a trace estimator

For any matrix H and any random vector v with

$$\mathbb{E}[vv^\top] = I,$$

we have

$$\mathbb{E}[v^\top Hv] = \mathbb{E}[\mathrm{tr}(v^\top Hv)] = \mathbb{E}[\mathrm{tr}(Hvv^\top)] = \mathrm{tr}\big(H\,\mathbb{E}[vv^\top]\big) = \mathrm{tr}(H).$$

This is the Hutchinson identity. It says that if the random probes are chosen in the right unbiased way, then the average quadratic form recovers the trace.

Rademacher probes satisfy exactly this requirement. If the entries of v are independent and each one is $+1$ or -1 with probability $1/2$, then

$$\mathbb{E}[v_i v_j] = \begin{cases} 1, & i = j, \\ 0, & i \neq j, \end{cases}$$

which is exactly the matrix statement

$$\mathbb{E}[vv^\top] = I.$$

So for Rademacher probes,

$$\mathbb{E}[v^\top H v] = \mathrm{tr}(H).$$

That is why the code samples random ± 1 directions rather than just picking a few fixed directions by hand.

8.5 Step 4: what the projector does to the trace

In the code the direction is not v, but Pv, where P removes the gauge subspace. Then the directional curvature becomes

$$(Pv)^\top H (Pv) = v^\top P^\top H P\, v.$$

Since the projector used here is orthogonal, $P^\top = P$ and $P^2 = P$, so the expectation becomes

$$\mathbb{E}[(Pv)^\top H (Pv)] = \mathrm{tr}(PHP).$$

So the stochastic sharpness quantity is not trying to estimate the trace of the full Hessian. It is trying to estimate the trace of the Hessian restricted to the non-gauge subspace.

Here the code applies the estimator not in the full space but in the gauge-projected subspace, so morally it is estimating the trace of the *physical* Hessian after removing two known symmetry directions.

8.6 What this scalar means intuitively

The trace is the sum of eigenvalues:

$$\mathrm{tr}(H) = \sum_{k=1}^{N} \lambda_k.$$

So the stochastic summary is approximately measuring the *sum of local curvatures* over all non-gauge directions.

This gives a useful but compressed notion of "sharpness":

- if many directions are strongly positively curved, the trace tends to be large and positive,
- if many directions are flat, the trace tends to be small,
- if positive and negative curvature coexist, they can cancel.

That last point is very important in this repo, because the Phase 13 Hessian analysis found indefinite saddle structure rather than clean positive-definite minima.

8.7 Why we want the trace at all

The trace is useful because it turns a high-dimensional curvature object into one understandable number. If the Hessian eigenvalues are

$$\lambda_1, \lambda_2, \ldots, \lambda_N,$$

then

$$\frac{1}{N}\operatorname{tr}(H) = \frac{1}{N}\sum_{k=1}^{N}\lambda_k$$

is the average curvature per direction.

In optimization language:

- a large positive trace means many random directions are bowl-shaped and steep,
- a smaller trace often means the point sits in a broader, flatter region,
- a flatter region is usually more robust to small numerical, physical, or modeling perturbations.

That is why sharpness regularization penalizes a trace-like quantity. It is a cheap way to say: "do not only make the loss small; also prefer solutions where the loss does not rise too quickly when the phase is nudged."

Key Idea

The sharpness path answers a different question: *"Can I add a cheap curvature penalty to the objective so the optimizer prefers flatter basins?"*

9 Why the Gauge Projector Appears in Sharpness

The Raman cost is invariant under two phase transformations on the input band:

$$\phi(\omega) \mapsto \phi(\omega) + C + \alpha\omega.$$

These correspond to:

- a constant phase shift,
- a linear-in-frequency phase shift, which acts like a time delay.

These are gauge directions: they do not change the physical cost. So if the sharpness estimator probed them directly, it would waste samples measuring known zero-curvature directions.

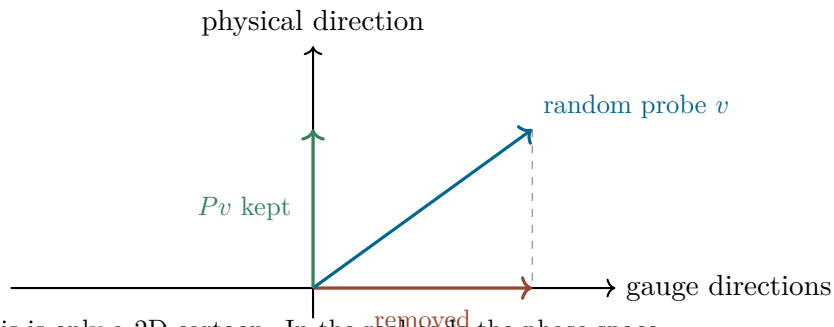
The code therefore constructs two orthonormal gauge vectors:

$$\begin{aligned} u_1 &= \text{constant-on-input-band direction}, \\ u_2 &= \text{centered linear-in-}\omega\text{ direction on the input band}, \end{aligned}$$

and projects by

$$P(v) = v - \langle u_1, v\rangle\, u_1 - \langle u_2, v\rangle\, u_2.$$

So all sharpness probes live in the complement of the gauge subspace.



This is only a 2D cartoon. In the real code the phase space is high-dimensional, and the projector removes the two known symmetry directions before measuring curvature.

Figure 4: Gauge projection removes directions that change phase convention but not the physical Raman cost.

10 How the Gradient of Sharpness is Computed

The code does not stop at estimating the scalar penalty $S(\phi)$. Since sharpness is used inside optimization, it also needs $\nabla S(\phi)$.

It computes this by applying the same symmetric finite-difference pattern to the gradient:

$$\nabla S(\phi) \approx \frac{1}{N_s} \sum_{i=1}^{N_s} \frac{\nabla J(\phi + \varepsilon P v_i) + \nabla J(\phi - \varepsilon P v_i) - 2\nabla J(\phi)}{\varepsilon^2}.$$

So the sharpness code reuses the existing first-order adjoint gradient oracle. It does *not* need a true second-order adjoint implementation.

If "second-order adjoint" is unfamiliar, you can read that sentence as: the code avoids building a much more complicated new derivative solver. It reuses the gradient code it already trusts.

10.1 What this buys the project

This is a practical engineering compromise:

- full second-order methods would be more exact but much more work,
- this estimator is cheaper to implement because it only reuses `cost_and_gradient(...)` repeatedly,
- the penalty can be dropped directly into L-BFGS as a modified scalar objective with a matching gradient.

11 The Actual Sharpness-Aware Objective

The code minimizes

$$J_{\text{sharp}}(\phi) = J_{\text{reg}}(\phi) + \lambda_{\text{sharp}}\, S(\phi),$$

with gradient

$$\nabla J_{\text{sharp}}(\phi) = \nabla J_{\text{reg}}(\phi) + \lambda_{\text{sharp}}\, \nabla S(\phi).$$

This is then passed to `Optim.LBFGS()` exactly like a normal objective.

11.1 Default settings

The Phase 14 implementation uses defaults:

- $\lambda_{\text{sharp}} = 0.1$,
- $N_s = 8$ Hutchinson samples,
- $\varepsilon = 10^{-3}$ for the sharpness finite difference.

By contrast, the Phase 13 HVP/eigenspectrum path uses a smaller finite difference default,

$$\varepsilon = 10^{-4},$$

because it is approximating the Hessian-vector product directly from gradient differences rather than a trace-like scalar penalty.

12 When Hessian is Used vs When Sharpness is Used

12.1 Hessian is used when the goal is geometry diagnosis

Use the Hessian path when you want to know:

- is the current point a minimum or a saddle,
- what are the stiff directions,
- what are the negative-curvature escape directions,
- how strong is the indefiniteness,
- what local second-order structure should guide future methods.

Operationally, this happens *after* an optimization run, not inside every L-BFGS iteration. In the Phase 13 path, it is a post-hoc analysis tool.

The exception is the separate trust-region research optimizer. That path uses finite-difference HVPs during optimization to build Newton-style steps, but it still avoids forming the full Hessian matrix.

12.2 Sharpness is used when the goal is to change the optimizer's behavior

Use the sharpness path when you want to say:

- do not just find a deep point,
- also prefer a locally flatter or more robust point,
- do this with the existing first-order infrastructure,
- do it during optimization, not only after the fact.

Operationally, sharpness is evaluated *inside* the objective at each optimization iteration.

Practical Takeaway

Hessian in this repo is primarily for *understanding* the landscape.
Sharpness in this repo is primarily for *biasing* the optimizer toward different parts of the landscape.

13 A Useful Conceptual Comparison

Question	Hessian path	Sharpness path
Primary role	diagnose local geometry	regularize optimization
Main object	$H(\phi)$ or its spectrum	$S(\phi)$, a trace-like scalar
Core computation	gradient finite difference for Hv	cost and gradient finite differences over random probes
Typical output	eigenvalues, eigenvectors, saddle/minimum verdict; or HVPs for trust-region steps	modified scalar objective and gradient
When used	usually post-hoc/offline; also inside the trust-region research optimizer	during sharpness-regularized optimization
Space used	full phase space or control space, depending on wrapper	physical phase space with gauge projection

14 An Important Subtlety: Signed Trace Cancellation

The research notes call out a key issue: the sharpness penalty implemented here is effectively a *signed* trace-like quantity.

If the Hessian is indefinite, then positive and negative curvature directions can cancel:

$$\mathrm{tr}(H) = \sum_i \lambda_i.$$

So a saddle with large positive and negative eigenvalues could still have a small trace.

Important Caveat

This means a small sharpness penalty does *not* necessarily mean the basin is truly flat in every direction. It may mean positive and negative curvature are canceling.

That is why the repo still keeps the explicit Hessian eigenspectrum machinery. The eigenspectrum tells you whether the geometry is genuinely flat, strongly stiff, or saddle-like. The sharpness penalty alone cannot always disambiguate those cases.

15 Control-Space vs Physical-Phase-Space Hessians

In the full-resolution problem, the optimization variable is the physical phase ϕ itself. But in reduced-basis problems the optimization variable may be a lower-dimensional coefficient vector x,

with

$$\phi = \phi(x).$$

Then the repo can define a Hessian oracle in control space by:

1. lifting x to the physical phase $\phi(x)$,
2. evaluating the physical loss and its gradient with respect to ϕ,
3. pulling that gradient back to the control variable.

This matters because the relevant geometry is the geometry of the variables the optimizer is actually moving, not always the larger full phase space in the background.

16 Why the Repo Chose These Implementations

The design logic is pragmatic:

1. The code already had a reliable first-order adjoint gradient.
2. A second-order adjoint would be a much larger implementation effort.
3. Finite-difference HVPs are expensive but manageable for diagnostics.
4. Hutchinson-style sharpness is noisy, but cheap enough to insert into an optimizer without rewriting the physics core.

So the codebase uses:

- **Hessian-vector products without forming the full Hessian** for exact-enough local geometry analysis and for the separate trust-region research optimizer,
- **stochastic sharpness estimation** for curvature-aware regularization during optimization.

17 A Clean Mental Model

It helps to keep the following hierarchy in mind:

Level 1: Base loss

$$J(\phi).$$

This is the physical or regularized Raman suppression cost.

Level 2: Gradient

$$\nabla J(\phi).$$

This comes from the forward-adjoint method already implemented in the main optimizer.

Level 3: Hessian-vector product

$$H(\phi)v.$$

This is approximated by finite-differencing the gradient oracle.

Level 4: Hessian eigenspectrum

$$\{\lambda_i, q_i\}.$$

This is extracted from the HVP operator using Arpack, usually only the top and bottom wings.

Level 5: Sharpness

$$S(\phi) \approx \text{average directional curvature over random projected directions.}$$

This is a compressed scalar summary of curvature, suitable for use as a regularizer.

Key Idea

The Hessian is the full local second-order object.
The sharpness penalty is a deliberately compressed, stochastic summary of that second-order object.

18 Final Summary

1. The repository's **Hessian** machinery is implemented through finite-difference Hessian-vector products built from the existing adjoint gradient oracle.
2. Those HVPs are used mainly for **post-hoc geometry analysis**: symmetry checks, Taylor tests, dense small-N verification, and top/bottom eigenspectrum extraction with Arpack. A separate trust-region research optimizer also uses finite-difference HVPs inside the optimization loop.
3. The repository's **sharpness** machinery is a **Hutchinson-style gauge-projected trace estimator** built from repeated cost and gradient evaluations.
4. Sharpness is used **inside optimization** by minimizing

$$J_{\text{sharp}} = J_{\text{reg}} + \lambda_{\text{sharp}} S.$$

5. Hessian and sharpness are related, but they serve different purposes: Hessian explains the local geometry; sharpness biases the optimizer using a cheaper summary of that geometry.
6. Because the implemented sharpness is trace-like and signed, it can hide cancellation between positive and negative curvature. That is why the explicit Hessian eigenspectrum remains important.

Practical Takeaway

If you only remember one sentence, remember this:
The Hessian code tries to understand the whole local shape of the landscape, while the sharpness code uses a random-sampling shortcut to boil part of that shape down to one number that can be used during optimization.